Quantum Path Finding Visualizer

Project Report

By **Abdullah Tariq**

This report summarizes the project structure, the path-finding algorithms used in the app, and the role of the quantum-inspired Grover approach. It discusses how the grovers search algorithm works in the context of path finding and how it compares to classical methods. It contains the analysis of the results, including a comparison table and visual outputs from the app.

April 29, 2026

Contents

1 Project Overview **2**

2 System Structure **2**

3 Algorithms **2**

3.1 Breadth-First Search 2

3.2 A* Search 3

3.3 Grover Search 3

3.3.1 Working of Grovers Algorithm 3

4 Visual Output **4**

5 Comparison Table **5**

6 Implementation Notes **5**

7 Conclusion **6**

1 Project Overview

The project has a Streamlit front-end and a Python back-end that implements three path-finding algorithms: Breadth-First Search (BFS), A* Search, and a quantum-inspired Grover search. The user can interact with the app to create a grid, place obstacles, and run the algorithms to find paths from a start node to an end node. Although I have provided random grid generation, the user can also manually edit the grid to create specific scenarios for testing.

The goal here is to compare the classical search methods (BFS and A*) with a quantum-inspired approach (Grover) in a way that is visually intuitive. The app focuses on clarity and side-by-side comparison rather than on heavy technical complexity, making it suitable for educational purposes and for demonstrating the differences between classical and quantum-inspired search methods.

2 System Structure

The codebase is divided into clear parts:

- **Grid management:** stores the board, start/end points, and obstacles.
- **Algorithms:** implements BFS, A*, and Grover-based path search.
- **Visualization:** renders the grid, highlights paths, and draws quantum plots.
- **UI layer:** handles the Streamlit layout, controls, and result display.
- **Utils:** contains helper functions for timing, metrics, and data formatting.

This structure keeps the app easy to maintain because each part has a specific responsibility. It also allows for future extensions, such as adding more algorithms or improving the quantum visualization, without needing to overhaul the entire codebase.

3 Algorithms

3.1 Breadth-First Search

BFS is a standard graph search method. It explores the grid level by level, so it is guaranteed to find the shortest path in an unweighted grid if one exists. In this project, BFS acts as a simple and reliable baseline for comparison.

At first it was giving same path as the Grovers search given that it uses it so I changed it slightly to have an ordered exploration of the neighbors (e.g., always checking right, down, left, up) to ensure it produces a consistent path that can be compared with the other algorithms.

3.2 A* Search

A* is one of the best-known informed search algorithms. It combines actual distance traveled with a heuristic estimate of the remaining distance to the goal. In a grid, the Manhattan distance is a natural heuristic because movement is restricted to nearby cells. A* is usually faster than BFS because it guides the search toward the target instead of exploring every direction equally.

This is well observed in the app, where A* typically explores fewer nodes than BFS, especially in larger grids with many obstacles.

3.3 Grover Search

Grover search is the quantum-inspired part of the project and is the most novel algorithm here. Unlike BFS and A*, which directly move through the grid, Grover search works on quantum states. It increases the probability of measuring a desired state by repeatedly amplifying it with Grover iterations.

In this app, Grover is used as a conceptual demonstration of how a quantum search process can support path finding. The algorithm is more experimental than BFS or A*, so the report should describe it as a new approach rather than a drop-in replacement for classical search. It acts as local search where it finds the path accurately with way fewer node explored.

3.3.1 Working of Grovers Algorithm

Grover's algorithm is designed to search an unsorted database with N entries in $O(\sqrt{N})$ time, which is a significant improvement over classical algorithms that require $O(N)$ time. In the context of path finding, we can think of the grid as a search space where each cell represents a potential state. Each cell here is encoded in a binary string, for example, in a 4×4 grid, we can represent each cell with a 4-bit binary string (e.g., 0000 for the top-left cell, 1111 for the bottom-right cell).

It encodes the number of qubits as $\lceil \log_2(N) \rceil$, where N is the total number of cells in the grid. It has 4 major steps:

1. **Initialization:** Start with an equal superposition of all possible states (cells).
2. **Oracle:** Apply a quantum oracle that marks the target state
3. **Diffusion Operator:** Apply the Grover diffusion operator to amplify the probability of the marked state.
4. **Convergence:** Measure the final state and check if it matches the target.

The algorithm iteratively applies the oracle and diffusion operator until it finds the target state with high probability. This approach is particularly effective in large search spaces,

making it an interesting choice for path finding in complex grids. This is usually done in $O(\sqrt{N})$ iterations, which is a significant improvement over classical search algorithms that require $O(N)$ time.

4 Visual Output

The app shows the grid, the discovered path, and supporting plots. For example, when run on a 8×8 grid, this is the grid produced alongside with the comparison of the algorithms and the grovers convergence plot.

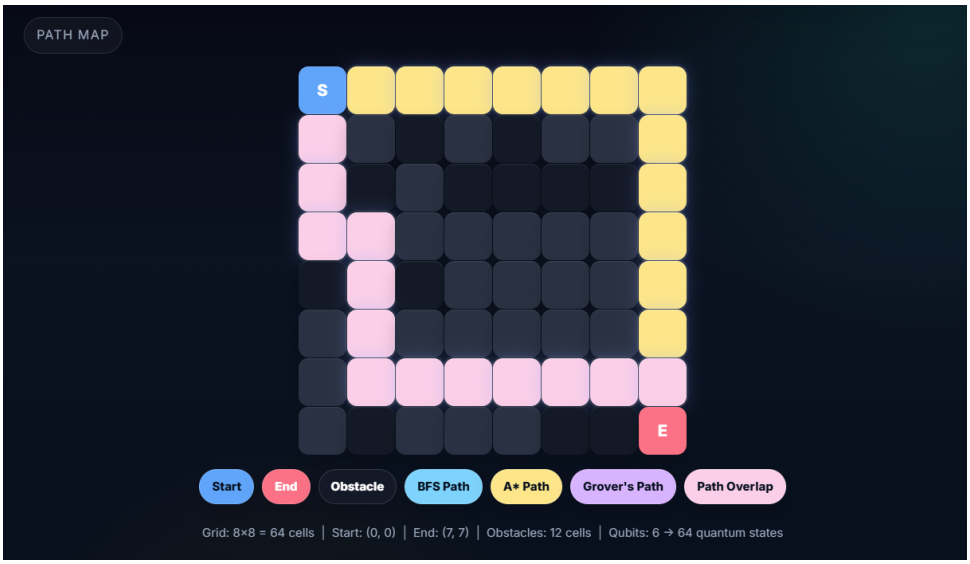

Figure 1: Example of the grid visualization showing the start point, end point, and obstacles.

Then the side-by-side comparison of the algorithms is shown, where you can see the time they took, nodes explored, and path length.

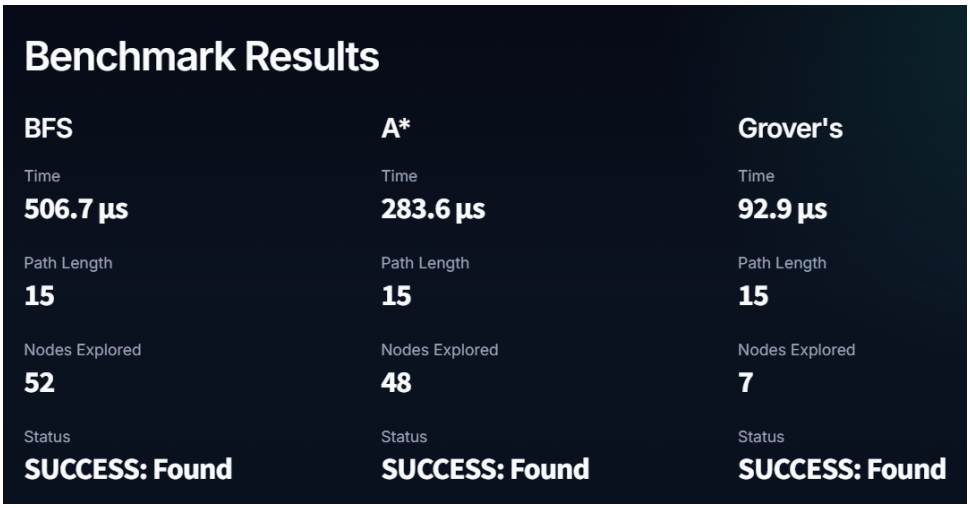

Figure 2: Side-by-side comparison of BFS, A*, and Grover search results.

Finally, the Grover-related plot shows how the probability of finding the target state evolves with each iteration, demonstrating the quantum search process.

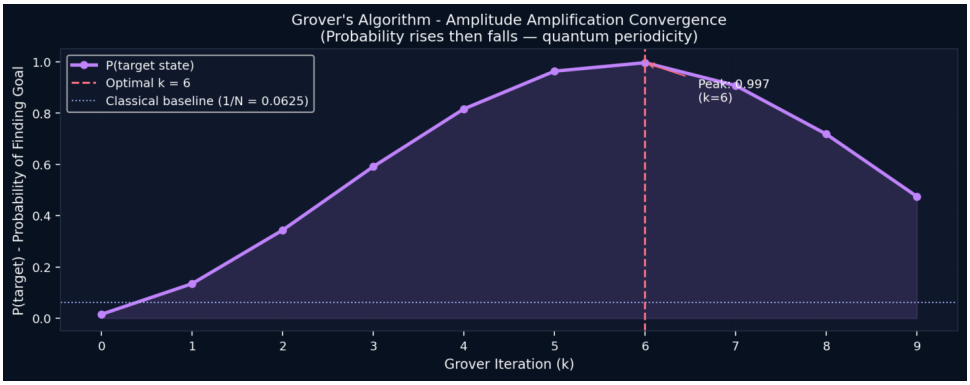

Figure 3: Grover's algorithm convergence plot showing the probability of finding the target state over iterations.

5 Comparison Table

This table summarizes the key metrics for each algorithm, including time taken, nodes explored, path length, and any relevant notes about their behavior in the context of this project.

Table 1: Algorithm Comparison Summary for $n = 16$

Algorithm	Time (ms)	Nodes Explored	Path Length	Efficiency	Notes
BFS	985.7µs	199	31	0.1558	Baseline shortest-path search.
A*	1.47ms	175	31	0.1771	Heuristic search using Manhattan distance.
Grover	72.7µs	13	31	2.3846	Quantum-inspired search and path reconstruction.

6 Implementation Notes

A few implementation choices make the app easier to use:

- The grid is editable so the user can place obstacles manually.
- The results are displayed in a consistent visual style.
- The UI separates input controls from output so the comparison is clear.

- The quantum visualizations are included as supporting material rather than as the main result.

7 Conclusion

This project provides a compact comparison between classical and quantum-inspired path finding. BFS offers a simple baseline, A* improves efficiency through heuristics, and Grover introduces a quantum-search perspective that makes the project more exploratory and research-oriented.

This table below summarizes the topics covered in this project and their relevance to the overall goal of demonstrating path finding algorithms in a visually intuitive way.

Course Topic	Implementation	Module
Introduction & Search Problem	The entire project is a live search problem on an NxN grid	app.py
Uninformed Search	BFS — explores level-by-level with no heuristic	algorithms/bfs.py
Informed Search	A* — uses Manhattan distance heuristic	algorithms/a_star.py
Local Search	Grover's iterative amplitude amplification (convergence plot)	algorithms/grovers_search.py
Adversarial Search	Obstacles simulate an adversarial environment blocking paths	grid/grid_manager.py
Constraint Satisfaction	Obstacles = hard constraints; Oracle encodes them in the quantum circuit	algorithms/grovers_search.py + grid/g
Probabilistic Reasoning	Quantum measurement is probabilistic; histogram visualizes uncertainty	visualization/quantum_plots.py
Knowledge Representation	Grid as explicit graph; cells as nodes; edges as valid moves	grid/grid_manager.py
Performance Measures	Time, path length, nodes explored, efficiency ratio compared side-by-side	utils/benchmarking.py

Figure 4: Topics covered in the project and their relevance to the overall goal.